

Homework 7

1. מחלקות ואובייקטים (Classes and Objects)

1.1. מה ההבדל בין מחלקה (Class) למופע (Instance/Object)? השתמש באנלוגיית ה"שרטוט".

מחלקה היא תבנית או הגדרה מופשטת, בעוד **שמופע** (או אובייקט) הוא התגלמות קונקרטית ושישה של אותה הגדרה.

תחשוב על מחלקה כעל **שרטוט** אדריכלי של בית. השרטוט מגדיר היכן ימוקמו הקירות, הדלתות והחלונות, אבל הוא אינו משהו שאפשר באמת לגור בו. מופע הוא הבית האמיתי והפיזי שנבנה באמצעות השרטוט הזה. אתה יכול לבנות בתים פרטיים רבים (מופעים) משרטוט בודד (מחלקה), ולמרות שכולם חולקים את אותו עיצוב מבני, הם ישויות פיזיות נפרדות לחלוטין.

1.2. מהו התפקיד של הפרמטר `self` במתודות של מחלקה?

הפרמטר `self` משמש כהפניה (Reference) לאובייקט הספציפי שקורא כעת למתודה. כאשר יש לך מספר אובייקטים שנוצרו מאותה מחלקה, `self` מבטיח שהמתודה תפעל על הנתונים של האובייקט הנכון. זהו המנגנון שמאפשר לאובייקט לגשת לתכונות (Attributes) ולמתודות של עצמו.

2. תכונות: מחלקה מול מופע (Attributes: Class vs. Instance)

2.1. מהי תכונת מחלקה (Class Attribute), וכיצד היא משותפת בין כל המופעים?

תכונת מחלקה היא משתנה שמוגדר ברמת המחלקה, מחוץ לכל מתודה ספציפית. מכיוון שהיא שייכת למחלקה עצמה ולא לאובייקט אינדיבידואלי כלשהו, היא משותפת לכל המופעים של אותה מחלקה. כל האובייקטים שנוצרים מהמחלקה הזו יצביעו בדיוק על אותו ערך בסיסי עבור התכונה הזו בזיכרון.

2.2. במה שונה תכונת מופע (Instance Attribute), והיכן היא מוגדרת בדרך כלל?

תכונת מופע היא משתנה ששייך בלעדית לאובייקט ספציפי אחד. כל מופע מתחזק עותק עצמאי משלו של התכונה, כלומר ששינויים בתכונה של אובייקט אחד לא ישפיעו על אובייקטים אחרים. תכונות מופע מוגדרות בדרך כלל בתוך מתודת הבנאי (Constructor) של המחלקה (מתודת האתחול) תוך שימוש בפרמטר `self` כדי לקשור את הנתונים ישירות לאובייקט שזה עתה נוצר.

2.3. מה קורה אם מנסים לשנות תכונת מחלקה דרך מופע ספציפי?

אם תנסה להקצות ערך חדש לתכונת מחלקה באמצעות אובייקט ספציפי, אתה לא באמת משנה את תכונת המחלקה המשותפת. במקום זאת, אתה יוצר בטעות **תכונת מופע** חדשה לחלוטין על אותו אובייקט ספציפי עם בדיוק אותו השם. תכונת המופע החדשה הזו "תסתיר" (Shadow) או תדרוס את תכונת המחלקה עבור אותו אובייקט ספציפי, אך כל שאר האובייקטים ימשיכו להצביע על תכונת המחלקה המקורית והבלתי משתנה.

3. זהות ושוויון (`is` מול `==`)

3.1. הסבר את ההבדל בין האופרטור `is` לאופרטור `==`.

- האופרטור `==` בודק שוויון של ערך. הוא בוחן האם הנתונים או התוכן בתוך שני אובייקטים שקולים זה לזה.
- האופרטור `is` בודק זהות. הוא בוחן האם שני משתנים מצביעים בדיוק על אותו אובייקט בזיכרון של המחשב.

3.2. כיצד ייתכן שלשני אובייקטים שונים יהיו אותם ערכים אך הם יחזירו `False` כאשר משווים אותם עם `is`?

אם תיצור שני אובייקטים נפרדים ותקצה להם נתונים זהים, יהיו להם את אותם הערכים (ולכן `==` יחזיר `True`). עם זאת, מכיוון שהם נוצרו כישויות נפרדות, המחשב מאחסן אותם במיקומים שונים לחלוטין בזיכרון. מכיוון שהאופרטור `is` מתייחס רק לכתובות זיכרון, השוואתם עם `is` תחזיר `False`, שכן הם אינם אותו אובייקט פשוטו כמשמעו.

4. מתודות קסם (`__eq__` ו-`__del__`)

4.1. מהי המטרה של המתודה `__eq__`, וכיצד היא מאפשרת לנו להתאים אישית את ההתנהגות של האופרטור `==`?

כברירת מחדל, כאשר אתה משתמש באופרטור `==` על אובייקטים מותאמים אישית (Custom Objects), פייתון לרוב רק בודקת אם הם בדיוק אותו אובייקט בזיכרון. המתודה `__eq__` מאפשרת לך לדרוס את התנהגות ברירת המחדל הזו ולהגדיר בדיוק מה המשמעות של "שוויון" עבור המחלקה הספציפית שלך. על ידי כתיבת לוגיקה פנימית בתוך `__eq__`, אתה מכתוב אילו תכונות יש להשוות כאשר משתמשים באופרטור `==` בין שני מופעים.

4.2. מתי נקראת המתודה `__del__` (הורס / Destructor), ומהו מקרה השימוש העיקרי שלה?

המתודה `__del__` מופעלת אוטומטית רגע לפני שאובייקט נהרס ומוסר לצמיתות מהזיכרון על ידי מנגנון "איסוף הזבל" (Garbage Collector) של המערכת (תהליך שקורה כאשר אין יותר משתנים שמצביעים על אותו אובייקט). מקרה השימוש העיקרי שלה הוא ביצוע פעולות ניקוי — כגון סגירת קבצים פתוחים, ניתוק חיבורים למסד נתונים, או שחרור משאבים חיצוניים שהאובייקט החזיק במהלך חייו.



```
nums = [1, 2, 4, 5, 5, 9, 10, 15]
```

```
# עד 20 של בטוח המספרים כל בדיקת ידי על הרשימה יצירת
```

```
close_numbers = [i for i in range(1, 21) if i in nums and (i + 1) in nums]
```

```
print(close_numbers)
```